

ITM-AE-DP-Blockwoche

DB-Benchmark

Ablauf des Blockwochenprojektes	2
Projektauftrag	2
Motivation mit Blick auf die Abschlußprüfung	2
Projektziel:	3
Beispiel für ein einfaches Projektziel	4
Projektschritte	5
Planung und Aufbau eines Testsystems	5
Genaue Formulierung des Szenarios	5
Einrichtung des Datenbanksystems	5
Aufstellung eines Testplanes	5
Ein einfaches Beispiel-Testsystem	6
Ein einfacher Beispiel Testplan	6
Durchführung der Messungen	7
Auswertung und Visualisierung der Ergebnisse	8
Überprüfung auf Kausalität und Messfehler	8
Ergänzende Untersuchungen	8
Visualisierungen und Schlussfolgerungen	8
Vorbereitung der Präsentation	9
Einführung in die Verwendung des JDBC	11
Datenbanken und das JDBC	11
Übungsaufgabe Rechnungserstellung	13
Arbeitsschritte	14
Aufgabe	14
Automatische Rechnungserstellung	15
Erweiterung um eine Bank	15
Aufgabe (Lösung durch ein Java-Programm)	15
Zusatzaufgabe: Gegenkonto	15
JDBC-Tutorial im Internet	16
Was ist JDBC?	16
Auf welchen Grundsätzen basiert die Verbindung zwischen Java und der Datenbank?	17
Einige wichtige Treiber herunterladen	21
Ein Projekt erstellen um das JDBC zu verwenden	22

Verbindungsaufbau	22
Vorschlag für eine Verbindungsklasse Db	23
Object Relational Mapping (ORM)	24
Das JDBC API zur Daten-Abfrage verwenden	27
Die Eigenschaften des ResultSet	27
Einführung in die Parallelprogrammierung	29
Nebenläufige Programmierung in Java	29
Die wichtigsten Methoden von Thread-Objekten	29
Automatisiertes Testen mit JUnit	36
Automatisch generierte Testklasse	37
Beispiel für eine Testklasse	38
Globale Testeinstellungen	38
Operationen vor und nach allen Tests	38
Operationen nach jedem Einzeltest	39
Auflistung der Einzeltests	39

Ablauf des Blockwochenprojektes

1-2 Wochentag	Einführung in die Verwendung des JDBC zur Verwendung von Datenbanken mit Java
2-3 Wochentag	Gruppen- und Themenfindung. Beschreibung von Testszenarios und Vermutungen von Ergebnissen
ab 3. Wochentag	Aufbau der Testszenarios. Durchführung und Dokumentation von Tests. Überprüfung der Ergebnisse Ergänzung bzw. Wiederholung von Tests Vorbereitung der Präsentationsinhalte
1-2 Woche nach den Ferien	Präsentation: - Fragestellung - Testszenario - Darstellung der Ergebnisse - Erkenntnisse

Projektauftrag

Motivation mit Blick auf die Abschlußprüfung

Das DB-Benchmark-Projekt hat einen direkten Bezug zu eurem späteren IHK-Abschlussprojekt. In beiden Fällen bearbeitet ihr ein frei wählbares fachliches Thema selbstständig und ihr durchlauft dabei alle wichtigen Projektphasen: Planung, Umsetzung, Auswertung und Präsentation. Im Projekt entwickelt ihr eine eigene Fragestellung, setzt ein technisches Szenario um und untersucht dieses systematisch – genau wie es auch im IHK-Projekt gefordert wird.

Dabei trainiert ihr wichtige Fähigkeiten, die ihr später unbedingt braucht. Dazu gehören strukturiertes Arbeiten, das Treffen und Begründen von Entscheidungen sowie die Auswertung von Ergebnissen. Auch die Dokumentation spielt eine zentrale Rolle: Ihr haltet eure Vorgehensweise und eure Erkenntnisse so fest, dass sie für andere nachvollziehbar sind. Das entspricht direkt den Anforderungen der Projektdokumentation bei der IHK.

Ein weiterer wichtiger Bestandteil ist die Präsentation. Ihr stellt eure Fragestellung, euer Vorgehen und eure Ergebnisse vor – genau wie später in der mündlichen Prüfung. So übt ihr nicht nur fachliche Inhalte, sondern auch, eure Arbeit verständlich und überzeugend zu präsentieren.

Insgesamt dient das Projekt also als Vorbereitung auf das IHK-Abschlussprojekt. Ihr könnt hier bereits die Arbeitsweise kennenlernen und wichtige Erfahrungen sammeln, die euch später helfen werden.

Projektziel:

„Ihr entwickelt ein System und untersucht anschließend dessen Performance.“

Ziel des Projektes ist es, im Rahmen der Blockwoche einer interessanten Fragestellung zum Thema DB - Benchmarking nachzugehen und hierzu eine Präsentation auszuarbeiten, die zu Beginn des nächsten Schuljahres präsentiert wird. Wählen Sie für die Anwendung einer Datenbank ein geeignetes Szenario und führen Sie Benchmarks aus, um z.B. folgende Fragestellungen zu beantworten.

- Welches Datenbankprodukt soll für eine bestimmte Anwendung eingesetzt werden?
- Welcher Einsatz von Rechnerressourcen ist sinnvoll (RAM, Cores, ...)?
- Welche Vernetzungsstruktur liegt vor? Welchen Einfluss hat das Netzwerk? (Mehrere Netzwerkkarten, Load-Balancing)
- Wie sollte der Zugriff mit Java programmiert werden (Statement, Prepared Statement, Batch Verarbeitung, Art des Resultsets, Transaktionsmodell, ...)
- Sollte ein dokumentenorientierter Ansatz gewählt werden (z.B. PostgreSQL, MongoDB)
- ...

„Wie beeinflussen verschiedene Parameter die Performance eines DB-Systems?“

Mögliche DB-Szenarios können sich in den folgenden Eigenschaften sehr unterscheiden:

1. Anzahl der Datensätze
2. Anzahl der Tabellen
3. Anzahl des Verhältnisses von Lese-, Schreib- und Änderungszugriffen
4. Art der abgelegten Daten (z.B. Typ und Größe)
5. Anzahl der konkurrierenden Zugriffe
6. Anzahl und Umgang mit Konflikten bei konkurrierenden Zugriffen
7. Limitierung der Ressourcen einer Datenbank (VM)
8. ...

Beispiel für ein einfaches Projektziel

In einem Kontensystem in einer Tabelle sollen innerhalb der Tabelle Kosten umgebucht werden. Es fallen sehr viele Buchungen auf relativ wenigen Konten (1000:1) an. Kontoüberziehungen außerhalb einer gewissen Toleranzgrenze sollen durch die Datenbank unterbunden werden. Es sollen ca. 90% der Buchungen erfolgreich sein. Als Datenbank soll HSQLDB eingesetzt werden.

Wie viel Arbeitsspeicher pro Konto sollte reserviert werden?

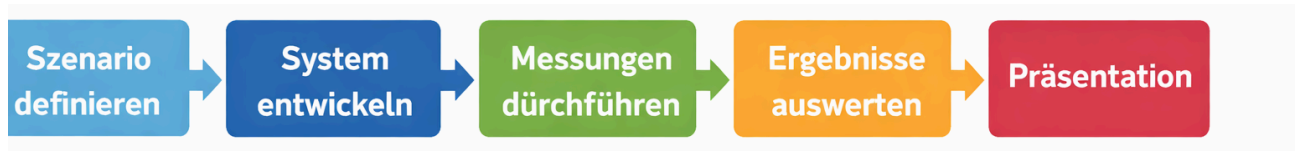
Wie sinkt die Ausführungsgeschwindigkeit bei Erhöhung der Buchungszahl oder der Kontenzahl?

Hauptsächlich soll die Frage beantwortet werden, wie der Zusammenhang zwischen Kontenzahl und der Buchungsgeschwindigkeit besteht, wenn ausreichend Speicher zur Verfügung steht. Der Einfluss von parallelem Zugriff und der Netzwerkanbindung soll hier nicht untersucht werden. Eine Optimierung der DB-Programmierung ist nicht erforderlich.

Projektschritte

„Wo ist der Weg zum Ziel?“

Meilensteine und Zuständigkeiten als Schlüssel zur erfolgreichen Planung!“



Diese allgemeine Vorgehensweise müssen Sie für sich im Rahmen dieses Projekts adaptieren.

Planung und Aufbau eines Testsystems

Genaue Formulierung des Szenarios

...

Einrichtung des Datenbanksystems

- Installation auf Hardware oder auf VM für Server und Clients.
- Erstellung der Testdatenbank
- Erstellung der Test-Clients

Aufstellung eines Testplanes

- Welche Messungen müssen durchgeführt werden?
- Welche Parameter werden konstant gehalten?
- Welche Parameter werden variiert?
- Sind die Messungen wiederholbar / Messwerter reproduzierbar (Einflussfaktoren)?
- Mit welchen Schwankungen bei der Messung oder bei der Übertragung auf ein zweites ähnliches System ist zu rechnen?

Ein einfaches Beispiel-Testsystem

Die Datenbank und der Test-Client werden innerhalb einer VM installiert. Die Speicher- und Prozessorzuteilung wird bei verschiedenen Testdurchläufen variiert. Der Test soll auf verschiedener Hardware ausgeführt und mindestens einmal ohne VM ausgeführt werden, damit untersuchungsfremde Einflüsse erkannt und minimiert werden können.

Testdatenbank und Test-Client können wie in dem Beispiel [Erweiterung um eine Bank](#) verwendet werden.

Ein einfacher Beispiel Testplan

Der in der Untersuchung wichtigste Zusammenhang ist der zwischen Buchungsgeschwindigkeit und Kontenzahl. Alle anderen Testparameter sollen so angepasst werden, dass ihr Einfluss minimiert wird. Es ist zu erwarten, dass die Größe des Arbeitsspeichers an die Anzahl der Konten angepasst werden muss, da HSQLDB nach Möglichkeit im Arbeitsspeicher arbeitet.

Um die Hardwareabhängigkeit zu beurteilen, werden die Messungen in einer VM auf zwei unterschiedlichen Systemen ausgeführt.

Die Transaktionsgeschwindigkeit in Abhängigkeit von der Kontenzahl soll auf folgenden Kombinationen von CPU-Cores und Arbeitsspeicher auf mindestens zwei verschiedenen Rechnern getestet werden:

	1024 MB	2048 MB	4096 MB	Native
1 Core	?? ms / 100%			----
2 Cores				----
4 Cores				----
Native	-----	-----	-----	

Die Messung der Ausführungszeiten soll durch den internen Millisekundentimer erfolgen. Die Anzahl der Buchungen wird so gewählt, dass die Messdauer im Minutenbereich liegt. Das Ergebnis wird in Transaktionen pro Sekunde angegeben. Die Ergebnisse sollen so normiert werden, dass die Ergebnisse auf den unterschiedlichen Rechner vergleichbar sind.

Durchführung der Messungen

Erfassung der Messungen und der Streuung.

Beispiel für einen Testplan

System 1) Intel Core i5 / 3500MHz / 4 Core

Datenbank und Testclient über localhost / Linux Mint Mate

1 / 1024 1000 Konten 10,9 MByte DB-File 50000 Transaktionen 27,67s, 31,72s, 27,55s 26,63s, 28,36s 2000 Konten 7,1 MByte 50000 Transaktionen 29,78s 27,67s 4000 Konten 24,0 MByte 50000 Transaktionen 27,23s 8000 Konten 33,4 MByte 50000 Transaktionen 27,5s 16000 Konten 44,4 MByte 50000 Transaktionen 27,6s ----- 32000 Konten 18,1 MByte 100000 Transaktionen 55,1s 64000 Konten 45,6 MByte 100000 Transaktionen 54,4s ----- 128000 Konten 47,7 MByte 400000 Transaktionen 215,3s 256000 Konten 76,9 MByte 400000 Transaktionen 218,9s	2 / 1024	4 / 1024	
1 / 2048	2 / 2048	4 / 2048	

1 / 4096	2 / 4096	4 / 4096	
			Native (ohne VM)

Die Abarbeitung solcher Testpläne sollte automatisch ablaufen. Damit die eigentlichen Ergebnisse über Nacht protokolliert werden können. Die Testprogramme sollten auch mit Zeitüberschreitungen und Abstürzen fertig werden. Die Verwendung von Unit Tests kann hier helfen:

Automatisiertes Testen mit JUnit

Plausibilitätsprüfung und Korrektur bei Fehlern.

Reduktion störender Einflussfaktoren.

Auswertung und Visualisierung der Ergebnisse

Überprüfung auf Kausalität und Messfehler

Untersuchung von Abhängigkeiten zwischen variablen Größen und Messergebnissen.

Bestimmung der Ordnung der Veränderung, wenn variable Parameter sich ändern.

Welche Parameter hängen stark voneinander ab?

Ergänzende Untersuchungen

Gibt es widersprüchliche Ergebnisse, wenn ja, wie kann man die Untersuchung verbessern?

Visualisierungen und Schlussfolgerungen

Ergebnisse von automatisierten Tests stehen in der Regel als CSV-Log-Datei zur Verfügung.

Beispiel:

```
11:33:52.371011100 Windows Native auf Linux-Mint-Native HSQLDB auf 192.168.4.55
LocalDateTime, DbDriver, DbUrl, t1, t2, Operation, KontenZahl, BuchungsZahl, Computer,
CPU, Cores, Takt, Ram, Result

2023-04-23T11:33:53.171093700,org.hsqldb.jdbcDriver,jdbc:hsqldb:hsqldb://192.168.4.55,10109
5506829800,101096294934500,anlegen,10,0,Linux Mint,i3,4,3.5,16384,done

2023-04-23T11:33:53.371865200,org.hsqldb.jdbcDriver,jdbc:hsqldb:hsqldb://192.168.4.55,10109
6298822900,101096496289800,buchen,10,100,Linux Mint,i3,4,3.5,16384,93

2023-04-23T11:33:53.898899700,org.hsqldb.jdbcDriver,jdbc:hsqldb:hsqldb://192.168.4.55,10109
6497378900,101097023340000,anlegen,20,0,Linux Mint,i3,4,3.5,16384,done

2023-04-23T11:33:54.561270300,org.hsqldb.jdbcDriver,jdbc:hsqldb:hsqldb://192.168.4.55,10109
7025860900,101097685692000,buchen,20,200,Linux Mint,i3,4,3.5,16384,192

2023-04-23T11:33:55.094760400,org.hsqldb.jdbcDriver,jdbc:hsqldb:hsqldb://192.168.4.55,10109
7686775000,101098219430300,anlegen,40,0,Linux Mint,i3,4,3.5,16384,done
```

```

2023-04-23T11:33:55.903305800,org.hsquidb.jdbcDriver,jdbc:hsquidb:hsquid://192.168.4.55,10109
8220639900,101099028039200,buchen,40,400,Linux Mint,i3,4,3.5,16384,382

2023-04-23T11:33:56.491701,org.hsquidb.jdbcDriver,jdbc:hsquidb:hsquid://192.168.4.55,10109902
9086700,101099615897500,anlegen,80,0,Linux Mint,i3,4,3.5,16384,done

2023-04-23T11:33:57.636059300,org.hsquidb.jdbcDriver,jdbc:hsquidb:hsquid://192.168.4.55,10109
9617293100,101100760495900,buchen,80,800,Linux Mint,i3,4,3.5,16384,743

2023-04-23T11:33:58.189865100,org.hsquidb.jdbcDriver,jdbc:hsquidb:hsquid://192.168.4.55,10110
0761704000,101101314898300,anlegen,100,0,Linux Mint,i3,4,3.5,16384,done

2023-04-23T11:33:59.198857700,org.hsquidb.jdbcDriver,jdbc:hsquidb:hsquid://192.168.4.55,10110
1315942100,101102323290900,buchen,100,1000,Linux Mint,i3,4,3.5,16384,955

2023-04-23T11:33:59.778844600,org.hsquidb.jdbcDriver,jdbc:hsquidb:hsquid://192.168.4.55,10110
2324327900,101102903273700,anlegen,200,0,Linux Mint,i3,4,3.5,16384,done

2023-04-23T11:34:02.194240100,org.hsquidb.jdbcDriver,jdbc:hsquidb:hsquid://192.168.4.55,10110
2904219300,101105319161900,buchen,200,2000,Linux Mint,i3,4,3.5,16384,1905

2023-04-23T11:34:02.937875400,org.hsquidb.jdbcDriver,jdbc:hsquidb:hsquid://192.168.4.55,10110
5320454300,101106062959200,anlegen,400,0,Linux Mint,i3,4,3.5,16384,done

2023-04-23T11:34:06.875886800,org.hsquidb.jdbcDriver,jdbc:hsquidb:hsquid://192.168.4.55,10110
6064295600,101110000728500,buchen,400,4000,Linux Mint,i3,4,3.5,16384,3816

2023-04-23T11:34:07.715343100,org.hsquidb.jdbcDriver,jdbc:hsquidb:hsquid://192.168.4.55,10111
0001989200,101110839773000,anlegen,800,0,Linux Mint,i3,4,3.5,16384,done

2023-04-23T11:34:15.737077,org.hsquidb.jdbcDriver,jdbc:hsquidb:hsquid://192.168.4.55,10111084
1199500,101118861504700,buchen,800,8000,Linux Mint,i3,4,3.5,16384,7584

2023-04-23T11:34:16.666407500,org.hsquidb.jdbcDriver,jdbc:hsquidb:hsquid://192.168.4.55,10111
8863049700,101119790825800,anlegen,1000,0,Linux Mint,i3,4,3.5,16384,done

2023-04-23T11:34:26.261533100,org.hsquidb.jdbcDriver,jdbc:hsquidb:hsquid://192.168.4.55,10111
9792203000,101129385961100,buchen,1000,10000,Linux Mint,i3,4,3.5,16384,9543

2023-04-23T11:34:27.950674500,org.hsquidb.jdbcDriver,jdbc:hsquidb:hsquid://192.168.4.55,10112
9387023400,101131075267200,anlegen,2000,0,Linux Mint,i3,4,3.5,16384,done

```

Derartige Dateien sind nicht gut interpretierbar und müssen daher aufbereitet werden:

- Diagramme von Messergebnissen mit Angabe der Streuung (Standardabweichung).
- Welche Aussagen kann man durch die Interpretation der Messergebnisse folgern?
-

Vorbereitung der Präsentation

Ergebnisse archivieren, da die bewertete Präsentation erst im neuen Schuljahr stattfindet.

In der Präsentation sollen die folgenden Punkte dargestellt werden:

- Fragestellung
- Szenario
- Ergebnisse (möglichst grafisch)

- Erkenntnisse
- Beschreibung des Testverfahrens, um die Gültigkeit der Ergebnisse zu belegen

Zur Vorbereitung der Präsentation werden in der Oberstufe noch einmal 2 Unterrichtsstunden bereitgestellt. Danach erfolgt eine Präsentation mit einer Dauer von ca. 20min.

Einführung in die Verwendung des JDBC

Datenbanken und das JDBC

Zugriff auf die HSQL-DB-Testdatenbank. Angelehnt an das Beispiel aus "Java ist auch eine Insel".

bis JDK 7	ab JDK 8 (Try with resources)
<pre>import java.sql.*; public class FirstSqlAccess { public static void main(String[] args) { try{ Class.forName("org.hsqldb.jdbcDriver"); } catch (ClassNotFoundException e){ System.err.println("Keine Treiber-Klasse!"); return; } Connection con = null; try { con = DriverManager.getConnection("jdbc:hsqldb:hsql://localhost", "SA", ""); Statement stmt = con.createStatement(); ResultSet rs = stmt.executeQuery("SELECT * FROM Customer"); while (rs.next()) System.out.printf("%s, %s %s\n", rs.getString("Id"), rs.getString(2), rs.getString(3)); rs.close(); stmt.close(); } catch (SQLException e) { e.printStackTrace(); } finally { if (con != null) try { con.close(); } catch (SQLException e) { e.printStackTrace(); } } } }</pre>	<pre>import java.sql.*; public class FirstSqlAccess { public static void main(String[] args) { try{ // kann unter JDK 8 entfallen Class.forName("org.hsqldb.jdbcDriver"); } catch (ClassNotFoundException e){ System.err.println("Keine Treiber-Klasse!"); return; } try (Connection con=DriverManager.getConnection("jdbc:hsqldb:hsql://localhost", "SA", ""); Statement stmt = con.createStatement()){ ResultSet rs = stmt.executeQuery("SELECT * FROM Customer"); while (rs.next()) System.out.printf("%d, %s %s\n", rs.getInt("Id"), rs.getString(2), rs.getString(3)); rs.close(); } catch (SQLException e) { e.printStackTrace(); } } }</pre>

Beide Varianten sind in der Funktion identisch. Alle Objekte, die das Interface “Closable” erfüllen, können als Argument eines Try-Statements erzeugt werden und werden dann in jedem möglichen Fall vor dem Verlassen des Try-Blocks wieder geschlossen (`AutoCloseable.close()`).

Der Aufruf von `executeQuery("SQL")` wird nur für SELECT-Anweisungen verwendet, da hier das Ergebnis ein Verweis auf eine Tabelle ist.

Für alle anderen Datenbankoperationen verwendet man `executeUpdate("SQL")`. Dieser Aufruf liefert die Anzahl der veränderten Datensätze zurück.

Der Datenbanktreiber “hsqldb.jar” liegt bei Linux-Systemen unter “/usr/share/java”.

Eine VM, die die notwendigen Entwicklungswerkzeuge integriert hat finden Sie auf:

`\\Synology_NAS\iservpublic\vms` im Schulnetz.

Login in die VM erfolgt mit info/geheim.

Übungsaufgabe Rechnungserstellung

Erstellen Sie ein Programm, das nach Eingabe einer Rechnungsnummer eine Rechnung für einen Kunden erstellt.

1 easybill GmbH • Düsselstr. 21 • 41564 Kaarst

3 **Herrn**
Max Mustermann
Muster Straße 1
12345 Musterstadt
Germany

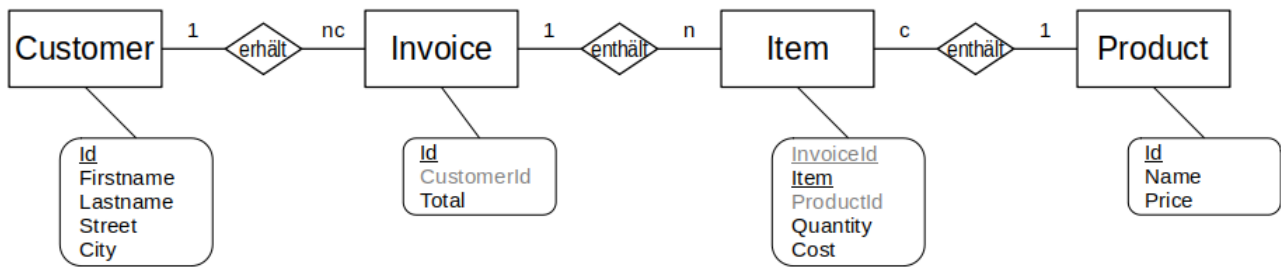
4 Datum **05.02.2020**
Kunde **10001**
Rechnung **2019266021**
Ihr Ansprechpartner **Max Mustermann**

Sehr geehrter Herr Mustermann,
nachfolgend berechnen wir Ihnen wie vereinbart:

5 **Rechnung 2019266021**
Das Rechnungsdatum entspricht dem Leistungsdatum 6

Pos	Bezeichnung	Menge	Einzelpreis netto	Betrag netto
7 1	Farbe Weiss - Eimer 20l	1	25,49	25,49 €
2	Streichen der Wände inkl. Abkleben	10	60,00	600,00 €
Nettobetrag				8 625,49 €
9 Umsatzsteuer 19%				118,84 € 10
Rechnungsbetrag				744,33 €

Die zur Verfügung stehende Datenbasis ist die Test-Datenbank von HSQLDB



Arbeitsschritte

- Kunde zur Rechnung ermitteln (Customer, Invoice)
- Anschreiben erstellen
- Rechnungsposten ermitteln (Product, Item)
- Rechnungsposten ausgeben
- Rechnungssumme ermitteln (Invoice, wenn nicht schon bekannt)
- Mehrwertsteuer berechnen und ausgeben

Aufgabe

Ermitteln Sie von Hand mit dem Datenbankmanager alle Rechnungsinformationen für die Rechnung Nr 13.

```
SELECT Id, Firstname, Lastname, Street, City
FROM Customer, Invoice
WHERE Customer.Id = Invoice.CustomerId
AND Invoice.Id = 13;
```

ID	FIRSTNAME	LASTNAME	STREET	CITY
6	Anne	Schneider	552 - 20th Ave.	Oltten

```
SELECT Item.Item, Name, Quantity, Cost, Quantity * Cost AS Betrag
FROM Item, Product
WHERE Product.Id = Item.ProductId
AND InvoiceId = 13;
```

ITEM	NAME	QUANTITY	COST	BETRAG
0	Telephone Ice Tea	18	22.50	405.00
1	Chair Clock	1	37.20	37.20
2	Clock Shoe	11	28.80	316.80
3	Shoe Iron	22	11.40	250.80
4	Iron Chair	21	32.70	686.70
5	Iron Clock	18	21.90	394.20
6	Iron Clock	22	21.90	481.80
7	Clock Ice Tea	11	33.90	372.90

Automatische Rechnungserstellung

Fragen Sie mit einer JOptionPane nach einer Rechnungsnummer und erstellen Sie eine HTML-Rechnung in einer Zeichenkette. Zeigen Sie diese Rechnung mit einer JOptionPane an.

Erweiterung um eine Bank

Die HSQL-GmbH hat eine Bank eingerichtet, um alle Kunden zu verwalten.

Die Rechnungen aller Kunden werden nur über die HSQL-Bank abgewickelt.

Die Bank verwendet die **Customer.Id** als Kontonummer.

Die Tabelle **Konto(Nr, Guthaben, Limit)** verwaltet die Kundeneinlagen.

Aufgabe (Lösung durch ein Java-Programm)

Erzeugen Sie einen neuen Kunden Bank.

Erzeugen Sie eine Tabelle "Konto".

Erzeugen Sie für alle Kunden eine Einlage zwischen 1000 und 5000 Euro.

Alle Kunden haben ein Limit von 1000.

Zusatzaufgabe: Gegenkonto

Legen Sie in der Kontentabelle ein Konto für die Bank an, indem die Summe aller Kundeneinlagen als negativer Wert abgelegt wird.

Die Summe aller Guthaben sollte anschließend den Wert 0,00€ haben. Wenn Umbuchungen

innerhalb der Bank durchgeführt werden, bleibt diese Summe immer 0,00€. Auf diese Weise lassen sich Fehler in Buchungsprogrammen leicht entdecken.

JDBC-Tutorial im Internet

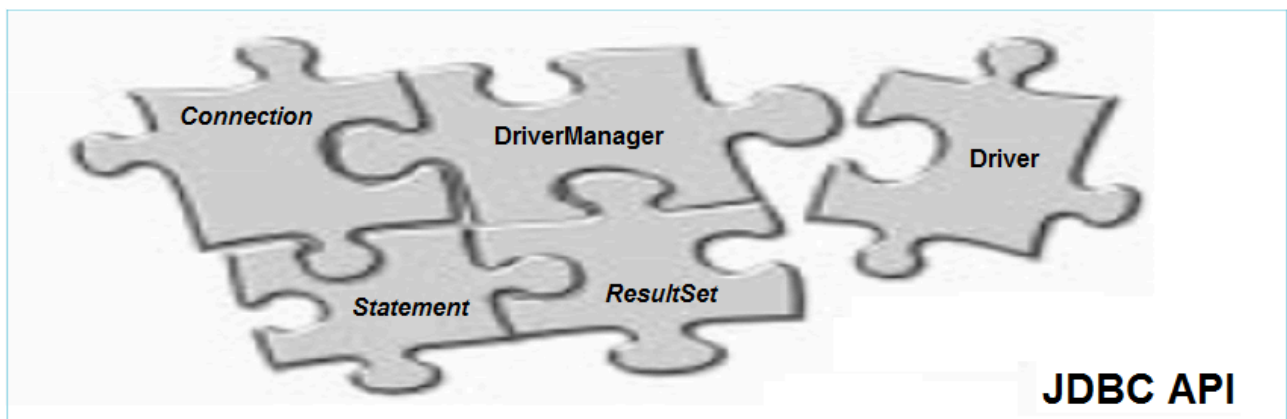
Hier finden Sie eine überarbeitete Kurzfassung des Internet-Tutorials:

[Die Anleitung zu Java JDBC](#)

Zusätzlich sind einige Vorschläge enthalten, um ihre Datenbankprogramme gut zu modularisieren.

Was ist JDBC?

JDBC (Java Database Connectivity) ist eine Standard-API zur Zusammenwirkung mit der Datenbank. **JDBC** hat eine Sammlung von Klassen und Interfaces, um die Datenbank anzusteuern.



Die grundlegenden Elemente der **JDBC API** sind:

1. **DriverManager:**
 - ist eine Klasse, die die Datenbanktreiber verwaltet.
2. **Driver:**
 - ist eine Implementierung für das Interface, das den Kontakt mit der Datenbank herstellt. Wenn der Treiber erfolgreich geladen wird, kann der Programmierer die in java.sql vorhandenen Konstrukte erfolgreich aufrufen. Der Name des Treibers ist in der Regel nicht gleich dem Namen der Treiberdatei und muss vom Datenbankanbieter in Erfahrung gebracht werden.

3. *Connection* :

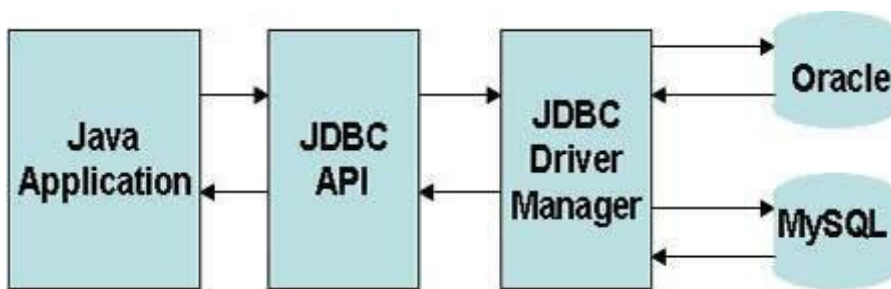
- ist ein Interface mit allen Methoden für die Verbindung mit der Datenbank. Alle Kontaktinformationen mit der Datenbank befinden sich im Connection-Objekt.

4. *Statement* :

- ist ein Interface für das Senden von SQL-Befehlen zur Datenbank. Es gibt neben dem Standard Statement auch noch spezialisierte Varianten (z.B. PreparedStatement, ...).

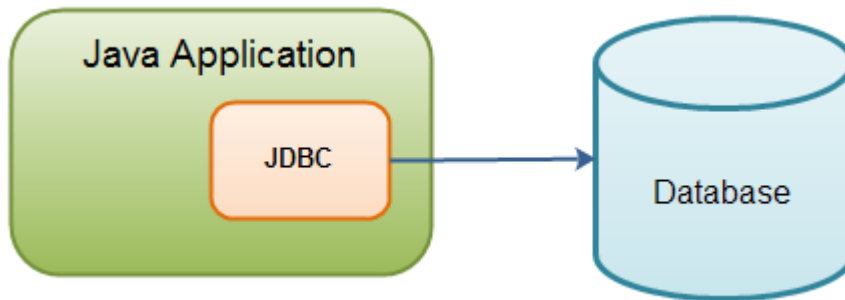
5. *ResultSet*:

- ist ein Zeiger auf die Antwort der Datenbank auf eine Abfrage. Ein Statement hat *genau ein* ResultSet. Die Eigenschaften des Resultsets können über das Statement eingestellt werden.



Auf welchen Grundsätzen basiert die Verbindung zwischen Java und der Datenbank?

Java benutzt das **JDBC**, um mit der Datenbank zu arbeiten.

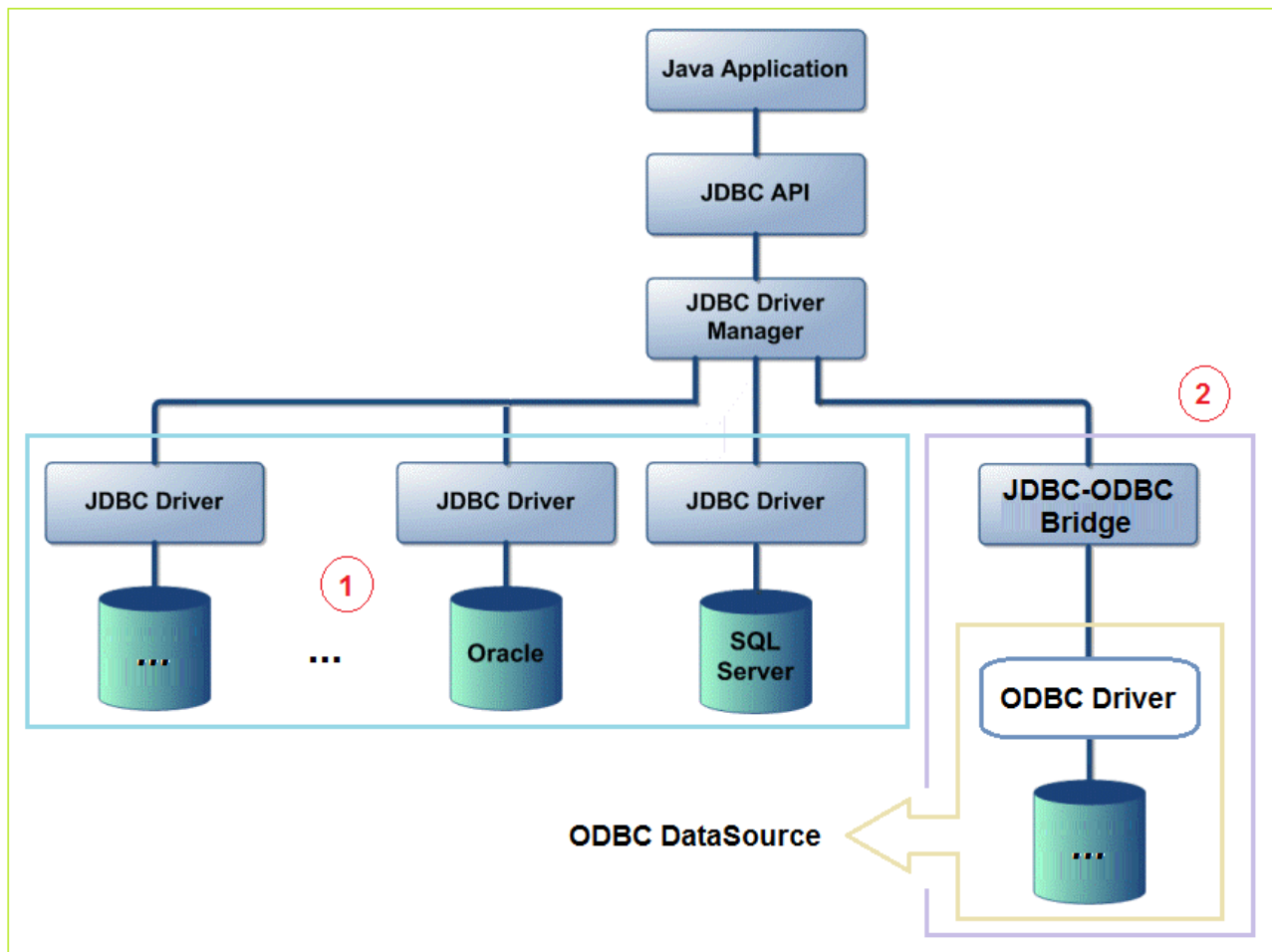


Zum Beispiel: Wenn Sie mit der Datenbank **Oracle** aus Java arbeiten, brauchen Sie einen Treiber (Das ist die Klasse für die Verbindung mit der Datenbank, die Sie möchten). In **JDBC API** haben wir *java.sql.Driver*, es ist aber nur ein Interface und es ist im **JDK** nicht implementiert. Deshalb müssen Sie den Treiber herunterladen, der der Datenbank entspricht.

- Wie: Mit **Oracle** ist die Implementierungclass der Interface *java.sql.Driver* :
oracle.jdbc.driver.OracleDriver

java.sql.DriverManager ist eine Klasse im **JDBC API**. Sie übernimmt die Aufgabe zur Verwaltung der Treiber

Sehen Sie die folgende Übersicht.



Wir haben heute 2 Möglichkeiten für den Umgang mit einer bestimmten Datenbank.

- M 1: Sie verwenden den Treiber für diese Datenbank. Das ist die direkte Maßnahme. Wenn Sie Oracle (oder eine andere DB) benutzen, müssen Sie den Treiber für diese DB herunterladen.
- M 2: Verwendung einer "**ODBC DataSource**", und Anwendung der Brücke **JDBC-ODBC**. Die Brücke **JDBC-ODBC** ist im **JDBC API** vorhanden



Data Sources
(ODBC)

Unsere Frage: Was ist "**ODBC DataSource**" ?

ODBC - Open Database Connectivity: Das ist ein Treiber für viele unterschiedliche Datenbanken, die eine gemeinsame, sehr einfache Schnittstelle verwenden. Diese Schnittstelle wird von Microsoft

bereitgestellt. Für diese Schnittstelle müssen die verwendeten ODBC-Treiber für die Datenbank aber innerhalb von Windows registriert werden.

ODBC DataSource: In dem Betriebssystem **Windows** können Sie eine Verbindung **ODBC** mit einer DB melden. Und dann haben wir eine Data Source.

Das **JDBC API** ist eine Brücke, mit dem das **JDBC** mit einer **ODBC Data Source** kommunizieren kann.

In Bezug auf die Geschwindigkeit ist die Möglichkeit 1 schneller als die Möglichkeit 2. Zusätzlich wird auch eine umfangreichere Funktionalität bereitgestellt. Die ODBC-Bridge erspart aber auf einem Anwendersystem oftmals die Einrichtung der Datenbankverbindung, wenn diese Verbindung schon für andere Systeme eingerichtet ist.

Einige wichtige Treiber herunterladen

Falls Sie die **JDBC-ODBC-Bridge** nicht benutzen möchten, können Sie die direkten Treiber für die Datenbank benutzen. In diesem Fall müssen Sie den jeder *DB* entsprechenden Treiber herunterladen. Hier sind die Dateinamen für die üblichen Datenbanken von

- **Oracle**
- **PostgreSQL**
- **MySQL**
- **MariaDB**
- **SQLServer**
- **HSQLDB**
-

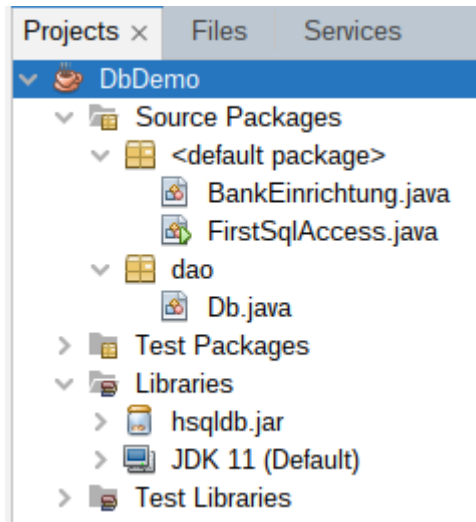
Sie können die Hinweise sehen bei:

- [JDBC Driver Bibliotheken für verschiedene Datenbanken in Java](#)

Database	Library	Treiberklassenname
Oracle	ojdbc6.jar	
PostgreSQL	postgresql-42.2.16.jar	org.postgresql.driver
MySQL	mysql-connector-java-x.jar	
MariaDb	mariadb-java-client.jar	org.mariadb.jdbc.Driver
SQL Server	jtds-x.jar	
	sqljdbc4.jar	
HSQLDB	hsqldb.jar	org.hsqldb.jdbcDriver

Ein Projekt erstellen um das JDBC zu verwenden

Ein Projekt neu erstellen;



Verbindungsaufbau

Um die Festlegung auf eine bestimmte Datenbank im Code zu isolieren, ist es zweckmäßig eine eigene Klasse für den Verbindungsaufbau zur Verfügung zu stellen, die den Zugriff auf eine bestimmte Datenbank, für einen bestimmten User mit einem bestimmten Datenbanktreiber kapselt:

Db
- usr - passwd - url - driver - con
+ getConnection() + close() + ...

Vorschlag für eine Verbindungsklasse Db

```
package dao;

import java.sql.*;

/**
 * Verbindungsklasse zur Datenbank. Jedes Db-Objekt verwaltet genau eine
 * Db-Verbindung. Kann auch mit try-with-resources verwendet werden.
 *
 * @author rLa
 */
public class Db implements java.lang.AutoCloseable {

    public String driver = "org.hsqldb.jdbcDriver";
    public String url = "jdbc:hsqldb:hsql://localhost";
    public String usr = "SA";
    public String passwd = "";

    private Connection con = null;

    /**
     * Baut eine Datenbankverbindung auf, wenn erforderlich
     *
     * @return
     */
    public Connection getCon() {
        // Treiber laden wenn erforderlich
        try { // kann unter JDK 8 entfallen
            Class.forName(driver);
            // Aufbau der Verbindung, wenn erforderlich
            if (con == null) { // Verbindung wurde noch nicht aufgebaut
                con = DriverManager.getConnection(
                    url, usr, passwd);
            } else if( !con.isValid(0)) { // Verbindung ist nicht mehr gueltig
                con = DriverManager.getConnection(
                    url, usr, passwd);
            }
        } catch (ClassNotFoundException | SQLException e) {
            System.err.println("Fehler: " + e + " beim Verbindungsaufbau.");
        }
        return con;
    }
}
```



```

/**
 * Schliesst die Datenbankverbindung, falls moeglich.
 *
 */
@Override
public void close() {
    if (con != null) {
        try {
            con.close();
        } catch (SQLException e) {
            System.err.println("Fehler: " + e + " beim Schliessen der Verbindung.");
        } finally {
            con = null;
        }
    }
}
}
}

```

Object Relational Mapping (ORM)

ORM = Verknüpfung von Datenbank-Tabellen und Java-Klassen.

Beispiel: Tabelle: Konto(Nr, Guthaben, Limit)

 Klasse: class Konto{

 // ... Methoden die mit Konto arbeiten

 }

Hier ein Vorschlag für die Customer-Tabelle:

```
package dao;

import java.sql.*;

/**
 * DAO-Klasse fuer die Tabelle Customer
 *
 * @author rla
 */
public class Customer {

    public Integer id;
    public String firstName;
    public String lastName;
    public String street;
    public String city;

    @Override
    public String toString() {
        return String.format(
            "%3d: %-15s %-15s %-20s %-15s",
            id, firstName, lastName, street, city);
    }

    public static Customer create(ResultSet rs) throws SQLException {
        Customer cu = new Customer();
        cu.id = rs.getInt("Id");
        cu.firstName = rs.getString("FirstName");
        cu.lastName = rs.getString("LastName");
        cu.street = rs.getString("Street");
        cu.city = rs.getString("City");
        return cu;
    }
}
```

Unser Einstiegsbeispiel wird damit zu:

```
import dao.Customer;
import dao.Db;
import java.sql.*;

public class DaoSqlAccess {

    public static void main(String[] args) {
        String sql = "";

        try ( Db db = new Db()) {
            Connection con = db.getCon();
            Statement stmt = con.createStatement();
            sql = "SELECT * FROM Customer";
            ResultSet rs = stmt.executeQuery(sql);

            while (rs.next()) {
                Customer cu = Customer.create(rs);
                System.out.println(cu);
            }
            rs.close();
            stmt.close();
        } catch (SQLException e) {
            System.err.println("Fehler " + e + " bei der Anweisung:\n" + sql);
        }
    }
}
```

Das JDBC API zur Daten-Abfrage verwenden

Die folgende Abbildung ist die Ansicht von Daten aus einer Tabelle **Employee**.

Sehen Sie ein Beispiel, wie **Java** auf die Daten zugreift:

```
Select Emp_Id, Emp_No, Emp_Name from Employee
```

	1	2	3
	EMP_ID	EMP_NO	EMP_NAME
next()	1	7839	E7839 ... KING ...
next()	2	7566	E7566 ... JONES ...
next()	3	7902	E7902 ... FORD ...
	4	7369	E7369 ... SMITH ...
	5	7698	E7698 ... BLAKE ...
	6	7499	E7499 ... ALLEN ...
	7	7521	E7521 ... WARD ...
	8	7654	E7654 ... MARTIN ...
	9	7782	E7782 ... CLARK ...
	10	7788	E7788 ... SCOTT ...
	11	7844	E7844 ... TURNER ...
	12	7876	E7876 ... ADAMS ...
	13	7900	E7900 ... ADAMS ...
	14	7934	E7934 ... MILLER ...

```
Connection connection = ....;

Statement statement =
    connection.createStatement();

ResultSet rs = statement.executeQuery(sql);

while (rs.next()) {
    int empId = rs.getInt(1);
    String empNo = rs.getString(2);
    String empName = rs.getString("Emp_Name");
}
```

ResultSet ist ein **Java** Objekt. Es wird zurückgegeben, wenn Sie die Daten abfragen (query). Verwenden Sie **ResultSet.next()** um den Cursor auf die nächsten Datensätze zu bewegen.

In einem Datensatz verwenden Sie die Method **ResultSet.getXxx()**, um den Wert in einer Spalte abzurufen. Die Spalte wird nach der Reihenfolge 1,2,3 ... markiert. Alternativ kann auch der Name der Spalte angegeben werden.

Die Eigenschaften des ResultSet

Sie kennen das **ResultSet** durch das obige Beispiel. Beim Auslesen der Daten kann ein **ResultSet** standardmäßig nur von oben nach unten und von links nach rechts durchlaufen werden.

Das bedeutet, dass Sie die folgenden Aufrufe nicht mit dem Default-**ResultSet** ausführen können.

- **ResultSet.previous()** : Einen Datensatz zurück.
- Auf dem gleichen Datensatz können Sie nicht **ResultSet.getXxx(4)** und anschließend **ResultSet.getXxx(2)** abrufen. (Geht bei fast jedem Nativ-JDBC-Treiber, ist aber nicht garantiert)

Ein solcher Aufruf bei einem Default-ResultSet kann eine **Exception** verursachen.

Mit einem angepassten createStatement-Aufruf können die Eigenschaften des Resultsets eingestellt werden.

resultSetType	Die Bedeutung
TYPE_FORWARD_ONLY	- ResultSet erlaubt nur von oben nach unten, von links nach rechts abzufragen. Dies ist der standardmäßige Typ vom ResultSet .
TYPE_SCROLL_INSENSITIVE	- ResultSet erlaubt vorwärts und rückwärts, rechts und links zu scrollen, aber ist nicht sensitiv mit die Änderungen der Daten. d.h während des Browsen eines Rekords und des Wiederbrowsen des Rekords nimmt es die Änderungen der Rekords nicht wahr.
TYPE_SCROLL_SENSITIVE	- ResultSet erlaubt vorwärts und rückwärts, rechts und links zu scrollen und ist sensitiv auf Änderungen der Daten während des Lesens.

resultSetConcurrency	Die Bedeutung
CONCUR_READ_ONLY	- Sie lesen nur die Daten beim Browsen der Daten mit diesem Typ von ResultSet
CONCUR_UPDATABLE	- Sie können nur die Daten bei der Leseposition ändern. Man nennt dies auch einen Read-Modify-Write-Zyklus. Wurde als besonders effizientes Mittel eingeführt, ist heute aber meist eher langsam. (Findet man nicht in der Literatur sondern durch Benchmarks)

Einführung in die Parallelprogrammierung

Nebenläufige Programmierung in Java

Um Datenbanktransaktionen zu testen, müssen parallele Zugriffe durchgeführt werden. Eine einfache Art solch ein Szenario auf einem Rechner zu erzeugen sind Java Threads.

Design von Programmen

Parallelprogrammierung mit Threads

Erstellung eines Threads



Für die Realisierung nebenläufiger Programmierung steht in Java die Klasse **Thread** zur Verfügung:

- Thread()
- Thread(String name)
- Thread(Runnable runnable)
- Thread(Runnable runnable, String name)

Anwendungsbeispiel:

```
class MyThread extends Thread {  
    public MyThread(String name) {  
        super(name);  
    }  
  
    public void run() {  
        for (int i = 0; i < 10; ++i) {  
            System.out.print "[" + this.getName() + i + " ] ";  
            try {  
                Thread.sleep(10);  
            } catch (InterruptedException ex) {  
                System.err.println("Thread wurde im Schlaf getötet!");  
            }  
        }  
    }  
}
```

ProgrammDesign.odp © 10.05.21 rla 13

Die wichtigsten Methoden von Thread-Objekten

1. **public void run():** wird ausgeführt wenn der Thread gestartet wird.
2. **public void start():** startet die run()-Methode des Threads im Hintergrund.
3. **public void sleep(long milliseconds):** Hält den Thread an, ohne Rechenzeit zu verbrauchen.
4. **public void join():** wartet bis die run()-Methode des Threads beendet ist (ohne Rechenzeit).
5. **public void join(long milliseconds):** wartet eine Zeit lang auf die Beendigung des Threads.
6. **public int getPriority():** liefert die Priorität des Thread.
7. **public int setPriority(int priority):** ändert die Priorität des Threads
8. **public String getName():** liefert den Namen des Thread.
9. **public void setName(String name):** ändert den Namen des Thread.
10. **public Thread currentThread():** liefert den Thread, der gerade ausgeführt wird.
11. **public int getId():** liefert die Id des Thread.
12. **public Thread.State getState():** liefert den Zustand des Thread.

13. **public boolean isAlive():** true, wenn die run()-Methode noch nicht beendet ist.
14. **public void yield():** wechselt sofort zum nächsten wartenden Thread.

Parallelprogrammierung mit Threads

Start eines Threads



Der Start eines Threads erfolgt mit **start()**. Dadurch wird der neue Thread gestartet. Der Programmteil der den Thread gestartet hat wird nicht unterbrochen.

```
MyThread thA = new MyThread("A");
MyThread thB = new MyThread("B");
thA.start();
thB.start();
```

```
run:
[A0] [B0] [B1] [A1] [B2] [A2] [B3] [A3] [B4] [A4] [B5] [A5] [A6] [B6] [A7] [B7] [A8] [B8] [A9] [B9]
```

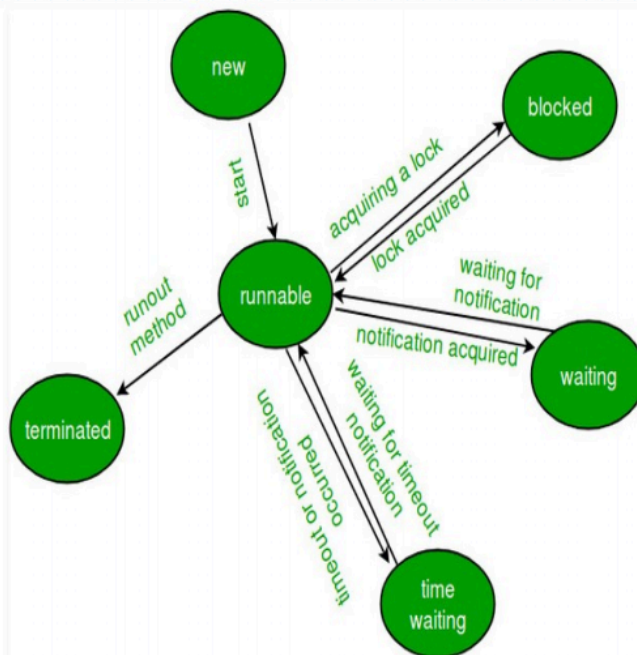
Wird dagegen **run()** aufgerufen findet keine nebenläufige Abarbeitung statt!

```
MyThread thA = new MyThread("A");
MyThread thB = new MyThread("B");
thA.run();
thB.run();
```

```
run:
[A0] [A1] [A2] [A3] [A4] [A5] [A6] [A7] [A8] [A9] [B0] [B1] [B2] [B3] [B4] [B5] [B6] [B7] [B8] [B9]
```

Parallelprogrammierung mit Threads

Zustände eines Threads



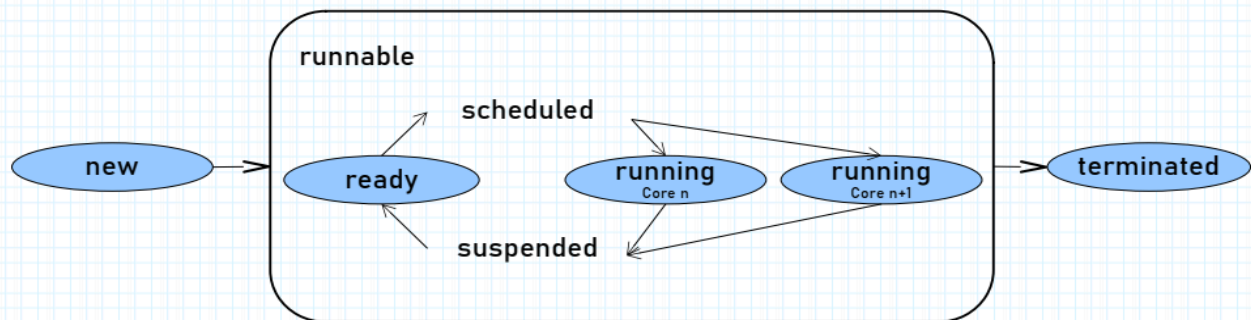
A thread state. A thread can be in one of the following states:

- **NEW**
A thread that has not yet started is in this state.
- **RUNNABLE**
A thread executing in the Java virtual machine is in this state.
- **BLOCKED**
A thread that is blocked waiting for a monitor lock is in this state.
- **WAITING**
A thread that is waiting indefinitely for another thread to perform a particular action is in this state.
- **TIMED_WAITING**
A thread that is waiting for another thread to perform an action for up to a specified waiting time is in this state.
- **TERMINATED**
A thread that has exited is in this state.

A thread can be in only one state at a given point in time. These states are virtual machine states which do not reflect any operating system thread states.

Die Art wie die Rechenzeit auf die Threads verteilt wird, ist nicht festgelegt.

Parallelprogrammierung mit Threads Rechenzeitverteilung



Im Idealfall wird die zur Verfügung stehende Leistung verteilt, in dem alle Threads für eine kurze festgelegte Zeitdauer von einem CPU-Core ausgeführt werden, bevor der nächste Thread zum Zuge kommt.

Durch die folgenden Aufrufe kann ein Thread auf einen Teil seiner Rechenzeit verzichten:

- Thread.yield() // Es wird zum nächsten Thread gewechselt
- Thread.sleep(ms) // Der Thread verbleibt eine Zeit im suspended Status

Zusätzlich kann ein Thread auch von außen in einen Wartezustand versetzt werden.

Parallelprogrammierung mit Threads

Das funktionale Interface Runnable



```

7  public class RunnableDemo {
8
9      public static void main(String... notUsed) {
10         Thread thA = new Thread(new Counter(), "A");
11         Thread thB = new Thread(new Counter(), "B");
12         thA.start();
13         thB.start();
14     }
15 }
16
17 class Counter implements Runnable {
18
19     @Override
20     public void run() {
21         for (int i = 0; i < 10; ++i) {
22             System.out.print "[" + Thread.currentThread().getName() + i + " ] ";
23             try {
24                 Thread.sleep(10);
25             } catch (InterruptedException ex) {
26                 System.err.println("Thread wurde im Schlaf gekillt!");
27             }
28         }
29     }
30 }

```

Wenn nicht von Thread abgeleitet werden soll, kann man als bessere Lösung das Interface **Runnable** verwenden. Es handelt sich um ein funktionales Interface. Es wird nur die parameterlose Methode **run()** vorgeschrieben.

Parallelprogrammierung mit Threads

Zugriff auf gemeinsame Ressourcen



```
18 public static void main(String... notUsed) {
19     SharedCounter job = new SharedCounter();
20     Thread th = new Thread(job, "A");
21     th.start();
22     for (int i = 0; i < 10; ++i) {
23         if (job.i == 3) {
24             System.out.println("Hallo");
25         }
26         pause(5);
27     }
28 }
29 }
30
31 class SharedCounter implements Runnable {
32
33     public volatile int i; // i steht mehreren Threads zur Verfügung
34
35     @Override
36     public void run() {
37         for (i = 0; i < 10; ++i) {
38             System.out.print "[" + Thread.currentThread().getName() + i + " ] ";
39             pause(10);
40         }
41     }
42 }
```

Mit volatile wird eine primitive Variable mehreren Threads zur Verfügung gestellt und werden nicht in den Prozessorcache optimiert.

Der Modifier volatile ist nur für primitive Variablen erlaubt.

Das Beispiel mit dem SharedCounter hätte man auch wie folgt realisieren können:

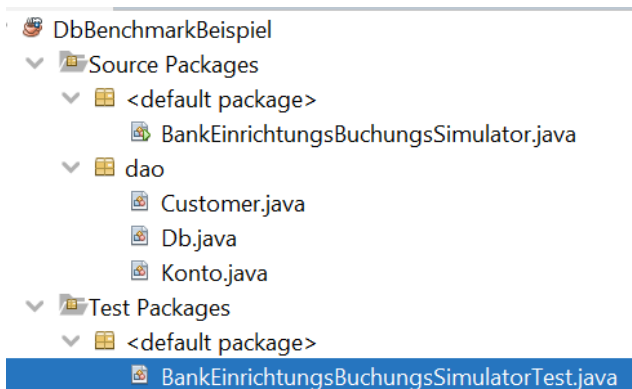
```
public synchronized int getI(){
    return i;
}
```

In der Collection Library existiert ein bekanntes Beispiel für vollständig vor Fehlern durch Parallelzugriff geschützte Objekte.

Dieser Schutz fordert aber seinen Preis. Vector ist deutlich langsamer als die HashList.

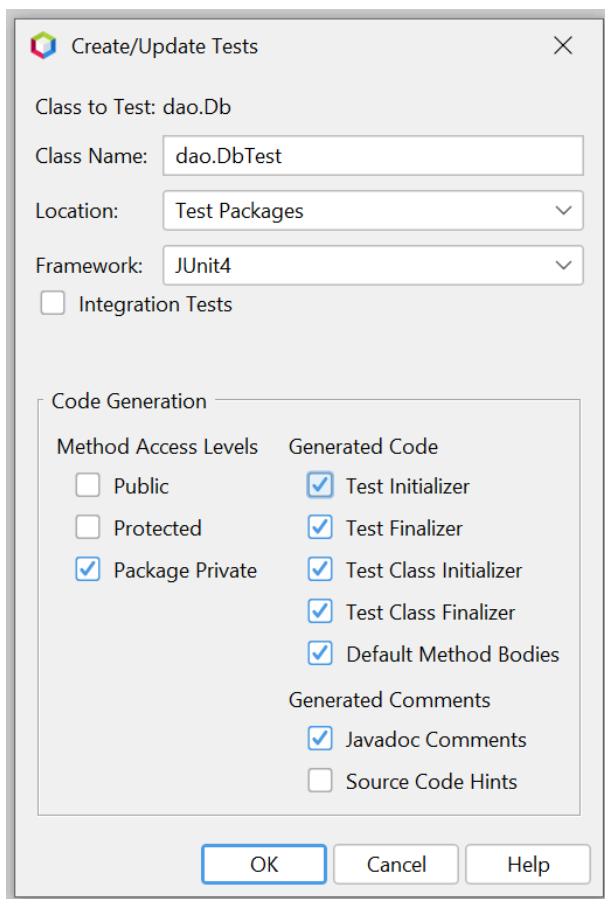
Automatisiertes Testen mit JUnit

Wenn viele Tests automatisiert durchgeführt werden sollen, ist JUnit ein gutes Hilfsmittel. JUnit ist gut in Netbeans integriert.



In diesem NetBeansprojekt gibt es einen Test, der ausgeführt wird, wenn im Menü Run die Option Test-Projekt aufgerufen wird.

Einen Test Prototyp kann Netbeans für jede Klasse im Source-Ordner automatisch erstellen.



Automatische Erstellung von Tests für die Klasse Db.java

Automatisch generierte Testklasse

```
package dao;

import org.junit.After;
import org.junit.AfterClass;
import org.junit.Before;
import org.junit.BeforeClass;
import org.junit.Test;
import static org.junit.Assert.*;

/**
 * Automatisch generierte Testklasse
 * @author rla
 */
public class DbTest {

    public DbTest() {

    }

    // Wird vor allen Tests aufgerufen
    @BeforeClass
    public static void setUpClass() {
    }

    // Wird nach allen Tests aufgerufen
    @AfterClass
    public static void tearDownClass() {
    }

    // Wird vor jedem Test aufgerufen
    @Before
    public void setUp() {
    }

    // Wird nach jedem Test aufgerufen
    @After
    public void tearDown() {
    }

    // Hier kommen die eigentlichen Testmethoden

    @Test
    public void testSomeMethod1() {
        fail("The test case is a prototype.");
    }

    @Test
    public void testSomeMethod2() {
        fail("The test case is a prototype.");
    }
}
```

```
}  
}
```

Beispiel für eine Testklasse

Globale Testeinstellungen

```
/**  
 * Automatischer Datenbanktest. Dieser Test geht davon aus,  
 * dass nur er auf die DB zugreift.  
 * Die Tests werden immer in alphabetischer Reihenfolge ausgeführt!  
 *  
 * @author rla  
 * @version 1.3  
 */  
@FixMethodOrder(MethodSorters.NAME_ASCENDING)  
public class BankEinrichtungBuchungsSimulatorTest {  
  
    public BankEinrichtungBuchungsSimulatorTest() {  
    }  
  
    private static Db db;  
    private static File logFile;  
    private static int kontenZahl;  
    private static int buchungsZahl;  
    private static long t1;  
    private static long t2;  
    private static String operation;  
    private static String result;  
    private static BufferedWriter logWriter;  
    private static String computer = "Linux Mint";  
    private static String cpu = "i3";  
    private static double takt = 3.5; // GHz  
    private static int core = 4;  
    private static int ram = 16384; // MByte
```

Operationen vor und nach allen Tests

```
// Wird vor den eigentlichen Tests durchgeführt.  
// Wenn mehrere Tests synchron gestartet werden sollen, kann man hier auf  
// den Startzeitpunkt warten.  
@BeforeClass  
public static void setUp() {  
    // Verbindung zur Testdatenbank vorbereiten.  
    // Hier die Verbindung zu einer Standard-HSQLDB-Installation auf Localhost
```

```

db = new Db();
db.driver = "org.hsqldb.jdbcDriver";
db.url = "jdbc:hsqldb:hsql://192.168.4.55";
db.usr = "SA";
db.passwd = "";

// Logfile vorbereiten
String txt = LocalTime.now()
    + " Windows Native auf Linux-Mint-Native HSQLDB auf 192.168.4.55" + "\n";
txt += "LocalDateTime, DbDriver, DbUrl, t1, t2, Operation, KontenZahl, BuchungsZahl,
Computer, CPU, Cores, Takt, Ram, Result\n";
logFile = new File("nativelog.csv");
if (logFile.exists()) {
    logFile.delete();
}
try {
    logWriter = new BufferedWriter(new FileWriter(logFile));
    logWriter.append(txt);
    logWriter.flush();
} catch (IOException ex) {
    System.err.println("Die log-Datei kann nicht geschrieben werden.");
}
}

// Logdatei nach allen Tests schliessen.
@AfterClass
public static void tearDown() {
    db.close();
    try {
        logWriter.close();
    } catch (IOException ex) {
        System.err.println("Die log-Datei konnte nicht geschlossen werden.");
    }
}

```

Operationen nach jedem Einzeltest

```

// Nach jedem Test wird das Ergebnis protokolliert.
@After
public void log() {
    log(logWriter, db, t1, t2, operation, kontenZahl, buchungsZahl,
        computer, cpu, core, takt, ram, result);
}

```

Auflistung der Einzeltests

```

// Hier kommen die eigentlichen Tests. Sie werden abgebrochen, wenn Sie

```



```

// laenger als 10min dauern. Alle Tests werden mehrmals ausgefuehrt, damit
// man die Schwankungen bei gleichen Parametern erkennt.
// Wenn ein Test durch einen Fehler abgebrochen wird, wird automatisch
// der naechste Test gestartet.
@Test(timeout = 600000L)
public void test001InitTestDb() {
    operation = "anlegen";
    buchungsZahl = 0;
    kontenZahl = 10;
    t1 = System.nanoTime();
    result = BankEinrichtungsbuchungsSimulator.initTestDb(db, kontenZahl);
    t2 = System.nanoTime();
    assertEquals("done", result);
}

@Test(timeout = 600000L)
public void test001runBuchungen() {
    operation = "buchen";
    buchungsZahl = kontenZahl * 10;
    t1 = System.nanoTime();
    int transaktionszahl = BankEinrichtungsbuchungsSimulator
        .runBuchungen(db, kontenZahl, buchungsZahl);
    t2 = System.nanoTime();
    result = "" + transaktionszahl;
    assertTrue(transaktionszahl > buchungsZahl / 2);
}

@Test(timeout = 600000L)
public void test002InitTestDb() {
    operation = "anlegen";
    buchungsZahl = 0;
    kontenZahl = 20;
    t1 = System.nanoTime();
    result = BankEinrichtungsbuchungsSimulator.initTestDb(db, kontenZahl);
    t2 = System.nanoTime();
    assertEquals("done", result);
}

@Test(timeout = 600000L)
public void test002runBuchungen() {
    operation = "buchen";
    buchungsZahl = kontenZahl * 10;
    t1 = System.nanoTime();
    int transaktionszahl = BankEinrichtungsbuchungsSimulator
        .runBuchungen(db, kontenZahl, buchungsZahl);
    t2 = System.nanoTime();
    result = "" + transaktionszahl;
    assertTrue(transaktionszahl > buchungsZahl / 2);
}

```

// ... ALLe gewuenschten Tests